

# ■ Smart Farming & Crop Planting Advisor

Full Codebase Exposition & Study Guide

Every file. Every function. Every design decision — explained in plain language.

Python + Streamlit Project | 2026

# Table of Contents

---

## 1. What Is This Project?

## 2. How the Project Is Organised (Folder Structure)

## 3. How the App Works — The Big Picture

## 4. File-by-File Deep Dive

4.1 `config.py` — Settings & API Keys

4.2 `data/crop_knowledge.json` — The Crop Database

4.3 `models/crop.py` — The Crop Class

4.4 `models/farm_plot.py` — The FarmPlot Class

4.5 `models/season_calendar.py` — Season Calendar & Weather Warnings

4.6 `services/weather_client.py` — Weather API Client

4.7 `services/planting_advisor.py` — The AI + Rule-Based Advisor

4.8 `services/farm_log_store.py` — Saving & Loading Data

4.9 `utils/validators.py` — Input Validation with Regex

4.10 `utils/logger_setup.py` — Logging System

4.11 `app.py` — The Streamlit User Interface

## 5. The Four Core Python Concepts Demonstrated

## 6. How All the Pieces Connect

## 7. The Data Flow Step-by-Step

## **8. Likely Exam / Presentation Questions & Answers**

# Chapter 1: What Is This Project?

---

The **Smart Farming & Crop Planting Advisor** is a web application built entirely in Python. A farmer types in their location and selects a crop, and the app tells them whether now is a good time to plant, what the weather looks like for the next 7 days, what to watch out for (pests, heatwaves, drought), and gives them a personalised calendar showing exactly when to weed and when to expect harvest.

It achieves this by pulling together three things at once:

1. **Baseline crop knowledge** — facts about each crop that never change, like 'maize needs 25mm of water per week' and 'tomatoes don't like temperatures above 27°C'. This is stored in a JSON file.
2. **Live weather data** — 7-day forecasts (rain, temperature, soil moisture) fetched in real time from the Open-Meteo API for the farmer's exact location.
3. **An AI or rule-based advisor** — combines the crop knowledge and the weather to generate a personalised recommendation. It tries Google's Gemini AI first; if that fails for any reason (no key, no internet), it silently falls back to a simpler logic engine so the app always gives an answer.

## Supported Crops

| Crop    | Planting Months                       | Growing Period | Water Needs/Week |
|---------|---------------------------------------|----------------|------------------|
| Maize   | March, April, May, August, September  | 100 days       | 25 mm            |
| Cassava | March–June, September, October        | 300 days       | 15 mm            |
| Tomato  | January, February, September–November | 80 days        | 30 mm            |
| Rice    | April, May, June, July                | 120 days       | 50 mm            |

## Tech Stack

| Technology                | What It Does in This Project                                 |
|---------------------------|--------------------------------------------------------------|
| Python 3.12               | Core programming language — all logic is in Python.          |
| Streamlit                 | Turns Python code into a web UI without needing HTML/CSS/JS. |
| requests                  | Makes HTTP calls to the weather API and geocoding APIs.      |
| google-genai              | The official Google SDK to call the Gemini AI API.           |
| python-dotenv             | Reads secret API keys from a .env file.                      |
| Open-Meteo API            | Provides free weather forecasts. No API key needed.          |
| OpenStreetMap / Nominatim | Backup geocoder (turns place names into coordinates).        |
| JSON files                | Stores crop knowledge and saved plot data on disk.           |

# Chapter 2: How the Project Is Organised

Understanding where each file sits tells you its purpose before you even open it. The project follows a clean separation: models hold data, services handle external communication and business logic, utils hold reusable tools, data holds static files, and app.py is the UI.

## Project Folder Tree

```
Smart-Farming-Advisor-main/
├──
│   ├── app.py                ← The Streamlit web UI (the face of the app)
│   ├── config.py            ← Loads .env keys and defines shared constants
│   ├── requirements.txt      ← Lists all Python packages to install
│   ├── .env                 ← Secret keys (GEMINI_API_KEY) – not in Git
│   ├──
│   │   ├── data/
│   │   │   ├── crop_knowledge.json ← Static crop facts (temp ranges, water needs,
│   │   │   │   pests)
│   │   │   ├── saved_plots/        ← Where farm_plots.json gets saved at runtime
│   │   │   │   ├── farm_plots.json ← Created when you click "Save all plots"
│   │   │   ├──
│   │   │   │   ├── models/
│   │   │   │   │   ├── crop.py      ← Crop class: loads and holds crop facts
│   │   │   │   │   ├── farm_plot.py ← FarmPlot class: one plot of land with its history
│   │   │   │   │   ├── season_calendar.py ← SeasonCalendar: milestones + weather warnings
│   │   │   │   │   ├──
│   │   │   │   │   ├── services/
│   │   │   │   │   │   ├── weather_client.py ← WeatherClient: fetches forecast from Open-Meteo
│   │   │   │   │   │   ├── planting_advisor.py ← PlantingAdvisor: Gemini AI + rule fallback engine
│   │   │   │   │   │   ├── farm_log_store.py ← FarmLogStore: reads/writes farm plots to disk
│   │   │   │   │   │   ├──
│   │   │   │   │   │   ├── utils/
│   │   │   │   │   │   │   ├── validators.py ← Regex-based input validation functions
│   │   │   │   │   │   │   ├── logger_setup.py ← Configures logging for all modules
```

**Rule of thumb:** If something talks to the internet or disk, it is in `services/`. If it is a blueprint for a thing (a crop, a plot), it is in `models/`. If it is a small reusable tool, it is in `utils/`. Everything is connected together in `app.py`.

# Chapter 3: How the App Works — The Big Picture

---

Before diving into individual files, here is the full journey of a user interaction from start to finish:

| S<br>t<br>e<br>p | What Happens                                                                                           | File Responsible                          |
|------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------|
| 1                | User types plot name, location, crop, and planting date into the form.                                 | app.py                                    |
| 2                | app.py calls <code>resolve_location()</code> to convert the location text into (lat, lon) coordinates. | app.py + validators.py                    |
| 3                | If it's a place name, the geocoding APIs are queried (Open-Meteo first, Nominatim as backup).          | app.py                                    |
| 4                | A <code>FarmPlot</code> object is created with the validated data and stored in session memory.        | models/farm_plot.py                       |
| 5                | User clicks 'Get weather & advice'. <code>WeatherClient</code> fetches a 7-day forecast.               | services/weather_client.py                |
| 6                | <code>Crop.load()</code> reads the crop's requirements from the JSON knowledge base.                   | models/crop.py + data/crop_knowledge.json |
| 7                | <code>PlantingAdvisor</code> combines crop knowledge + weather and calls Gemini for advice.            | services/planting_advisor.py              |
| 8                | If Gemini fails (no key, no internet), rule-based fallback logic produces advice instead.              | services/planting_advisor.py              |

| S<br>t<br>e<br>p | What Happens                                                                       | File Responsible           |
|------------------|------------------------------------------------------------------------------------|----------------------------|
| 9                | SeasonCalendar generates milestone dates and scans the forecast for risk warnings. | models/season_calendar.py  |
| 10               | All results are displayed in the Streamlit UI.                                     | app.py                     |
| 11               | User clicks 'Save all plots'. FarmLogStore serialises the plots to JSON on disk.   | services/farm_log_store.py |
| 12               | Next time the app opens, FarmLogStore.load() reads those plots back automatically. | services/farm_log_store.py |



# Chapter 4: File-by-File Deep Dive

---

## 4.1 config.py — Settings & API Keys

---

**What this file does:** This is the very first thing that runs when the app starts. Its only job is to read the secret API key from a file called {c('.env')} and make a small set of shared constants available to the rest of the project.

**Why a separate file?** Putting settings in one place means if you ever change the save directory or the cache duration, you change it once here rather than hunting through five different files.

*config.py — complete*

```
import os
from dotenv import load_dotenv

load_dotenv()  # reads the .env file and loads its key=value pairs into the
environment

GEMINI_API_KEY      = os.getenv("GEMINI_API_KEY")
WEATHER_CACHE_MINUTES = 60          # cached weather is reused for 60 minutes
SAVE_DIRECTORY      = "data/saved_plots"
SUPPORTED_CROPS     = ["maize", "cassava", "tomato", "rice"]

def validate_config():
    if not GEMINI_API_KEY or GEMINI_API_KEY == "your_gemini_api_key_here":
        raise ValueError("GEMINI_API_KEY is missing.")
```

### Key concepts demonstrated:

- **os.getenv()** — reads an environment variable. If the variable does not exist it returns None.
- **load\_dotenv()** — reads the .env file and loads its contents so os.getenv() can find them.
- **validate\_config()** — a 'fail fast' guard. The app calls this at startup. If the key is missing it shows an error immediately instead of crashing later with a confusing API error.

**Important:** The .env file is never committed to Git (it is listed in .gitignore). This keeps your secret key out of public repositories.

## 4.2 data/crop\_knowledge.json — The Crop Database

**What this file does:** This JSON file is the static knowledge base for the app. It stores agricultural facts about each of the four supported crops. Nothing in this file is ever changed while the app runs — it is reference data, like a textbook.

*crop\_knowledge.json — maize entry (example)*

```
{
  "maize": {
    "display_name": "Maize",
    "planting_months": [3, 4, 5, 8, 9],
    "water_needs_mm_per_week": 25,
    "growing_period_days": 100,
    "ideal_temp_min_c": 18,
    "ideal_temp_max_c": 32,
    "common_pests": ["fall armyworm", "stem borer"],
    "common_diseases": ["maize streak virus", "leaf blight"],
    "drought_sensitive": true,
    "heat_sensitive": false
  }
}
```

| JSON Key                | What It Means                                         | How It's Used                                                         |
|-------------------------|-------------------------------------------------------|-----------------------------------------------------------------------|
| display_name            | The human-readable crop name.                         | Shown in UI labels and advice text.                                   |
| planting_months         | List of month numbers (1=Jan) when planting is ideal. | Crop.is_in_planting_season() checks if current month is in this list. |
| water_needs_mm_per_week | How much rain/irrigation the crop needs weekly.       | Used in the Gemini prompt to contextualise rainfall data.             |
| growing_period_days     | Days from planting to expected harvest.               | SeasonCalendar adds this to planting date to predict harvest day.     |

| JSON Key                 | What It Means                              | How It's Used                                                                |
|--------------------------|--------------------------------------------|------------------------------------------------------------------------------|
| ideal_temp_min_c / max_c | Temperature comfort zone for the crop.     | SeasonCalendar checks if forecast exceeds max_c to trigger heatwave warning. |
| common_pests / diseases  | Known threats to this crop.                | Included in AI prompt and rule-based fallback text.                          |
| drought_sensitive        | True if low rain severely harms this crop. | Triggers irrigation warning in rule-based fallback.                          |
| heat_sensitive           | True if high temperatures harm this crop.  | Triggers heat warning in rule-based fallback.                                |

## 4.3 models/crop.py — The Crop Class

**What this file does:** Defines the `crop` class. A `Crop` object is a container that holds all the facts about one crop (from the JSON file) and provides one useful method: checking whether a given month is inside the planting season.

**Why a class and not just read the JSON directly?** Wrapping the data in a class gives you one place to put methods that work ON that data (like `is_in_planting_season`), and makes the code that uses it readable: you write `crop.is_in_planting_season(month)` instead of repeating `month` in `all_crops[crop_name]["planting_months"]` everywhere.

*models/crop.py — load() classmethod*

```
@classmethod
def load(cls, crop_name: str, filepath: str = "data/crop_knowledge.json"):
    crop_name = crop_name.lower().strip()

    try:
        with open(filepath, "r") as f:
            all_crops = json.load(f)  # read the entire JSON file into a dictionary
    except FileNotFoundError:
        raise FileNotFoundError(f"Crop knowledge base not found at '{filepath}'.")
    except json.JSONDecodeError:
        raise ValueError("Crop knowledge base is corrupted or not valid JSON.")
```

```

if crop_name not in all_crops:
    raise ValueError(f"'{crop_name}' is not a supported crop.")

data = all_crops[crop_name]
return cls(name=crop_name, **data) # ** unpacks the dict as keyword arguments

```

### Breaking down every line:

- **@classmethod** — this decorator means the method belongs to the class itself, not to an instance. You call it as `Crop.load('maize')` without creating a `Crop` first. The first parameter is `cls` (the class), not `self`.
- **with open(filepath, "r") as f** — opens the JSON file for reading. Using 'with' means Python automatically closes the file when the block ends, even if an error occurs.
- **json.load(f)** — parses the entire JSON file into a Python dictionary.
- **except FileNotFoundError** — catches the specific error Python raises if the file path does not exist. Raises it again with a clearer message.
- **except json.JSONDecodeError** — catches corrupted or invalid JSON files.
- **return cls(name=crop\_name, \*\*data)** — creates and returns a new `Crop` object. The `**` operator unpacks the dictionary into keyword arguments, so `data['display_name']` becomes `display_name=...`, etc.

*models/crop.py — is\_in\_planting\_season()*

```

def is_in_planting_season(self, month: int) -> bool:
    return month in self.planting_months
    # If planting_months = [3, 4, 5, 8, 9], then:
    # is_in_planting_season(4) -> True
    # is_in_planting_season(6) -> False

```

**\_\_repr\_\_:** The `__repr__` method controls what Python prints when you inspect the object in the console. Without it you get `.` With it you get `Crop(Maize)` — much easier during debugging.

## 4.4 models/farm\_plot.py — The FarmPlot Class

**What this file does:** Defines the `FarmPlot` class. A `FarmPlot` object represents one piece of land the farmer is managing. It stores the plot's name, GPS coordinates, the crop being grown, the planting date, and a timestamped activity log.

**Key design decision:** FarmPlot is a pure data class. It stores information but does NOT call any API or make any decisions. Other classes (PlantingAdvisor, SeasonCalendar) operate ON a FarmPlot. This is the Single Responsibility Principle: each class does one thing.

*models/farm\_plot.py — init and log activity*

```
def __init__(self, plot_name, latitude, longitude, crop_name, planting_date=None):
    self.plot_name      = plot_name
    self.latitude       = latitude
    self.longitude      = longitude
    self.crop_name      = crop_name
    self.planting_date  = planting_date    # optional: None until farmer sets it
    self.activity_log   = []              # starts empty: a list of dicts

def log_activity(self, activity: str, date=None):
    if date is None:
        date = datetime.now()    # defaults to right now
    self.activity_log.append({"date": date, "activity": activity})
```

### Serialisation — to\_dict() and from\_dict():

JSON files cannot store Python objects or datetime values directly. The to\_dict() method converts the FarmPlot into a plain dictionary (with dates as strings) so it can be saved. from\_dict() does the reverse when loading back from disk.

*models/farm\_plot.py — to\_dict()*

```
def to_dict(self) -> dict:
    return {
        "plot_name":      self.plot_name,
        "latitude":       self.latitude,
        "longitude":      self.longitude,
        "crop_name":      self.crop_name,
        "planting_date": (
            self.planting_date.strftime("%Y-%m-%d") # datetime -> "2026-08-01"
string
            if self.planting_date else None
        ),
        "activity_log": [
            {
                "date":      entry["date"].strftime("%Y-%m-%d"),
                "activity": entry["activity"],
            }
            for entry in self.activity_log    # list comprehension iterates over all
entries
        ],
    }
```

The **list comprehension** [...for entry in self.activity\_log] is a concise way to loop over activity\_log and build a new list in one line, equivalent to a for-loop that appends to an empty list.

## 4.5 models/season\_calendar.py — Season Calendar & Weather Warnings

**What this file does:** The `SeasonCalendar` class takes a `FarmPlot` and a `Crop` and produces two things: (1) a list of milestone dates, and (2) weather warnings by scanning the forecast for dangerous conditions.

*models/season\_calendar.py — generate\_milestones()*

```
def generate_milestones(self) -> list:
    planting_date = self.farm_plot.planting_date
    return [
        {"date": planting_date, "milestone": "Planting"},
        {"date": planting_date + timedelta(days=14), "milestone": "First
weeding"},
        {"date": planting_date + timedelta(days=28), "milestone": "Second
weeding"},
        {"date": planting_date + timedelta(days=self.crop.growing_period_days),
         "milestone": "Expected
harvest"},
    ]
```

**timedelta:** imported from Python's datetime module. Adding `timedelta(days=14)` to a date shifts it forward by exactly 14 days, handling month boundaries and leap years automatically.

### Weather Warning Logic:

The `check_weather_warnings()` method scans the 7-day forecast for three types of risk:

| Risk Type             | Condition                                | Threshold Constants |
|-----------------------|------------------------------------------|---------------------|
| Heavy Rain / Flooding | Any single day exceeds 40mm of rainfall. | HEAVY_RAIN_MM = 40  |

| Risk Type           | Condition                                                          | Threshold Constants                            |
|---------------------|--------------------------------------------------------------------|------------------------------------------------|
| Dry Spell / Drought | 3 or more consecutive days with less than 1mm of rain.             | DRY_SPELL_DAYS = 3, DRY_SPELL_THRESHOLD_MM = 1 |
| Heatwave            | 2 or more consecutive days above the crop's ideal max temperature. | HEATWAVE_DAYS = 2                              |

*season\_calendar.py — dry spell detection (streak counter pattern)*

```
dry_streak = 0 # reset the counter
for date, rain in zip(dates, rainfall):
    if rain < DRY_SPELL_THRESHOLD_MM: # this day counts as 'dry'
        dry_streak += 1
        if dry_streak == DRY_SPELL_DAYS: # threshold reached – warn once
            warnings.append(f"Dry spell warning: 3+ consecutive days ending
{date}")
    else:
        dry_streak = 0 # a wet day resets the streak
```

**zip(dates, rainfall)** — pairs each date with its matching rainfall value, so you can loop over them together. zip([1,2,3], ['a','b','c']) -> [(1,'a'), (2,'b'), (3,'c')].

## 4.6 services/weather\_client.py — Weather API Client

**What this file does:** The `WeatherClient` class is responsible for talking to the Open-Meteo API. It fetches a 7-day weather forecast for any GPS coordinates and handles caching so the same request is not made repeatedly.

### Caching — Why and How:

Every time a user clicks 'Get weather & advice', Streamlit reruns the entire script. Without caching, every click would fire a new HTTP request to the API. The cache stores results in a dictionary inside the object. Each entry has a timestamp. When a new request arrives, it checks the age — if less than 60 minutes old it returns the cached version instantly.

*weather\_client.py — cache check logic*

```

cache_key = (round(latitude, 2), round(longitude, 2), forecast_days)

cached = self._cache.get(cache_key)    # look up the dictionary
if cached:
    fetched_at, data = cached
    age_minutes = (datetime.now() - fetched_at).total_seconds() / 60
    if age_minutes < WEATHER_CACHE_MINUTES:    # still fresh
        return data                            # return stored result

# Cache miss or expired – make the actual HTTP request
response = requests.get(BASE_URL, params=params, timeout=10)

```

## Exception Handling Pattern:

The requests library raises its own specific exception types. This file catches each one and re-raises it as a simpler, cleaner Python built-in type. This means app.py only needs to catch three well-known types — not five different requests-specific ones.

| requests Exception                  | Converted To    | Meaning                          |
|-------------------------------------|-----------------|----------------------------------|
| requests.exceptions.Timeout         | TimeoutError    | Server took too long to respond. |
| requests.exceptions.ConnectionError | ConnectionError | No internet or DNS failure.      |
| requests.exceptions.HTTPError       | ValueError      | Server returned 4xx/5xx error.   |

## The Forecast Dictionary Structure:

After a successful call, `_parse_response()` converts the raw API JSON into a clean dictionary that the rest of the app uses:

*forecast dict structure*

```

{
    "dates":      ["2026-07-30", "2026-07-31", ...], # 7 strings
    "temp_max":   [32.1, 29.4, ...],                 # daily high temperatures in Celsius
    "temp_min":   [22.0, 20.8, ...],                 # daily low temperatures
    "rainfall_mm": [5.2, 0.0, ...],                  # daily precipitation totals in mm
    "soil_moisture": 0.35                             # current topsoil moisture (or None)
}

```



## 4.7 services/planting\_advisor.py — The AI + Rule-Based Advisor

---

**What this file does:** The `PlantingAdvisor` class is the decision engine. It takes a `Crop` object and a weather forecast and produces a planting recommendation. It tries Google Gemini first. If Gemini fails for ANY reason, it falls back to an offline rule-based engine.

### The Gemini Path:

When Gemini is available, the advisor builds a detailed text prompt containing the crop's baseline data and the forecast data, sends it to Gemini, and returns Gemini's response as the advice text.

*planting\_advisor.py — the prompt that gets sent to Gemini*

```
f"You are an agricultural advisor. A farmer wants to grow {crop.display_name} "
f"this month (month {current_month}).\n\n"
f"Crop baseline data:\n"
f"- Ideal planting months: {crop.planting_months}\n"
f"- Water needs: {crop.water_needs_mm_per_week} mm/week\n"
f"- Ideal temperature range: {crop.ideal_temp_min_c}-{crop.ideal_temp_max_c} C\n"
f"- Common pests: {'', '.join(crop.common_pests)}\n\n"
f"Upcoming weather forecast:\n"
f"- Average daily rainfall: {avg_rainfall:.1f} mm\n"
f"- Temperature range: {min_temp}-{max_temp} C\n"
f"- Current topsoil moisture: {soil}\n\n"
f"In 3-4 short sentences, advise the farmer..."
```

### The Fallback Path:

If Gemini raises any exception, the broad except clause catches it and calls `_rule_based_advice()` instead. This method compares numbers directly:

*planting\_advisor.py — rule-based fallback logic*

```
def _rule_based_advice(self, crop, weather, current_month):
    avg_rainfall = sum(weather["rainfall_mm"]) / len(weather["rainfall_mm"])
    max_temp     = max(weather["temp_max"])
    in_season    = crop.is_in_planting_season(current_month)

    notes = []

    if not in_season:
```

```

        notes.append(f"This month is outside {crop.display_name}'s planting
window.")

    if crop.heat_sensitive and max_temp > crop.ideal_temp_max_c:
        notes.append(f"Forecast highs of {max_temp}C exceed tolerance. Consider
shade or delay.")

    if crop.drought_sensitive and avg_rainfall < 5:
        notes.append("Low rainfall – irrigation will likely be needed.")

    if not notes:    # no problems found
        notes.append(f"Conditions look reasonable for {crop.display_name}.")

    return {"source": "rule-based fallback", "should_plant_now": in_season,
"advice_text": " ".join(notes)}

```

**Why catch all exceptions broadly here?** In most code, catching all exceptions with bare 'except Exception' is bad practice because it hides bugs. Here it is deliberately correct: whatever goes wrong with Gemini (bad key, rate limit, network error, API change), the response is identical — use the fallback. So there is no benefit in distinguishing the error types.

## 4.8 services/farm\_log\_store.py — Saving & Loading Data

**What this file does:** The `FarmLogStore` class handles all file I/O. It converts a list of `FarmPlot` objects to JSON and writes them to disk, and does the reverse when loading back.

*farm\_log\_store.py — save()*

```

def save(self, plots: list) -> None:
    os.makedirs(self.directory, exist_ok=True)    # create folder if it doesn't
exist
    data = [plot.to_dict() for plot in plots]    # convert each FarmPlot to a dict

    try:
        with open(self.filepath, "w") as f:
            json.dump(data, f, indent=2)    # write formatted JSON to disk
    except OSError as e:
        raise OSError(f"Could not save farm plots: {e}")

```

*farm\_log\_store.py — load()*

```
def load(self) -> list:
    if not os.path.exists(self.filepath): # no file = first run, not an error
        return []

    try:
        with open(self.filepath, "r") as f:
            data = json.load(f)
    except json.JSONDecodeError as e:
        raise ValueError("Saved file is corrupted.")

    return [FarmPlot.from_dict(entry) for entry in data]
```

**os.makedirs(directory, exist\_ok=True)** — creates the save folder and all its parent folders if they do not exist. The exist\_ok=True flag prevents an error if the folder is already there (without it, Python would raise FileExistsError on every save after the first).

**indent=2 in json.dump()** — makes the output JSON file human-readable with 2-space indentation instead of one long unbroken line.

## 4.9 utils/validators.py — Input Validation with Regular Expressions

**What this file does:** Contains functions that check user input is correct before it is used anywhere else. Every function raises a ValueError with a specific error message if the input is bad.

**What is a Regular Expression (regex)?** A regex is a pattern that describes what a string should look like. Python's re module matches strings against patterns. re.compile() pre-builds the pattern once so it runs faster on repeated calls.

| Function               | Regex Pattern                                                   | What It Accepts                                                                 |
|------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------|
| validate_coordinates() | <code>^\s*(-?\d+(?:\.\d+)?)\s*,\s*(-?\d+(?:\.\d+)?)\s*\$</code> | '9.0765, 7.3986' — optional minus, digits, optional decimal, comma, same again. |

| Function                 | Regex Pattern                                    | What It Accepts                                                         |
|--------------------------|--------------------------------------------------|-------------------------------------------------------------------------|
| validate_location_name() | ^[A-Za-z\s\-',\,]+\s\$                           | Letters, spaces, hyphens, apostrophes, commas, periods only. No digits. |
| validate_date()          | ^(\d{4})-(0[1-9] 1[0-2])-(0[1-9] [12]\d 3[01])\$ | '2026-08-15' format. Month must be 01–12, day must be 01–31.            |
| extract_numeric_value()  | -?\d+\.\d*                                       | Pulls the first number from any string. '23.5mm' -> 23.5.               |

*validators.py — coordinate validation (annotated)*

```

COORDINATE_PATTERN = re.compile(
    r'^\s*(-?\d+(?:\.\d+)?)\s*,\s*(-?\d+(?:\.\d+)?)\s*$'
    #   ^      ^      ^^              ^      ^^              ^
    # start |   | optional  comma |                               end
    #       |   | decimal part      |
    #   optional                               second number (longitude)
    #   minus sign
)

def validate_coordinates(coord_string: str) -> tuple[float, float]:
    match = COORDINATE_PATTERN.match(coord_string)
    if not match:
        raise ValueError(f"'{coord_string}' is not valid 'latitude, longitude'
format.")

    lat, lon = float(match.group(1)), float(match.group(2))

    if not (-90 <= lat <= 90):    # Earth latitude is -90 to +90
        raise ValueError(f"Latitude {lat} is out of range.")
    if not (-180 <= lon <= 180): # Earth longitude is -180 to +180
        raise ValueError(f"Longitude {lon} is out of range.")

    return lat, lon

```

**match.group(1) and match.group(2)** — refers to the first and second capturing groups (the parts inside parentheses) in the pattern. Group 1 is the latitude, group 2 is the longitude.

## 4.10 utils/logger\_setup.py — Logging System

---

**What this file does:** Configures Python's built-in logging module so every part of the app can record what it's doing. Every module calls `setup_logger(__name__)` to get a logger named after that file.

**Why logging instead of print()?** `print()` statements disappear when the terminal closes. Logs are saved to `logs/app.log` permanently. Each log message is automatically timestamped and labelled with severity (INFO / WARNING / ERROR) and which module produced it.

*utils/logger.py — setup\_logger()*

```
def setup_logger(name: str) -> logging.Logger:
    os.makedirs("logs", exist_ok=True)    # ensure logs/ folder exists

    logger = logging.getLogger(name)      # reuse existing logger if already created
    logger.setLevel(logging.INFO)         # minimum severity level to record

    if not logger.handlers:               # avoid duplicate handlers on Streamlit
reruns
        formatter = logging.Formatter(
            "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
            datefmt="%Y-%m-%d %H:%M:%S"
        )

        file_handler = logging.FileHandler("logs/app.log")    # writes to disk
        console_handler = logging.StreamHandler()             # prints to
terminal

        file_handler.setFormatter(formatter)
        console_handler.setFormatter(formatter)

        logger.addHandler(file_handler)
        logger.addHandler(console_handler)

    return logger
```

The guard `if not logger.handlers` is important for Streamlit specifically: Streamlit reruns the whole script every time a user clicks anything. Without this guard, a new handler would be added on every rerun, causing every log message to be printed multiple times.

---

## 4.11 app.py — The Streamlit User Interface

**What this file does:** This is the entry point and the glue layer. It imports every other class and function in the project and wires them together into a working user interface.

## Streamlit Fundamentals:

**How Streamlit works:** Every time a user interacts with the page (clicks a button, changes a dropdown, types in a field), Streamlit reruns app.py from top to bottom. This means any variable you create is re-created from scratch on every click. To keep things like the WeatherClient (which holds the cache) alive between clicks, you store them in st.session\_state.

*app.py — session\_state initialisation (runs only once per browser session)*

```
if "weather_client" not in st.session_state:
    st.session_state.weather_client = WeatherClient()    # create once, reuse
    forever

if "advisor" not in st.session_state:
    st.session_state.advisor = PlantingAdvisor()

if "store" not in st.session_state:
    st.session_state.store = FarmLogStore()

if "plots" not in st.session_state:
    st.session_state.plots = st.session_state.store.load() # load saved plots on
    startup
```

The `if "key" not in st.session_state` pattern means: 'if this object does not exist yet (first run), create it'. On every subsequent rerun, the condition is False and the object is reused.

## Geocoding — resolve\_location():

The location input field accepts both place names ('Wuse, Abuja') and raw coordinates ('9.0765, 7.3986'). The `resolve_location()` function decides which path to take:

*app.py — resolve\_location()*

```
def resolve_location(location_input: str) -> tuple:
    if any(char.isdigit() for char in location_input):
        return validate_coordinates(location_input)    # coordinates path

    clean_location = validate_location_name(location_input)
    return geocode_location(clean_location)            # place-name geocoding path

def geocode_location(location_name: str) -> tuple:
    try:
```

```

        return _geocode_open_meteo(location_name)      # try primary geocoder
    except Exception:
        return _geocode_nominatim(location_name)      # fall back to Nominatim

```

## Section 1 — Adding a New Farm Plot:

*app.py — new plot form*

```

with st.form("new_plot_form"):
    plot_name      = st.text_input("Plot name", placeholder="e.g. North Field")
    location_input = st.text_input("Location", placeholder="e.g. Wuse, Abuja or
9.07, 7.39")
    crop_choice    = st.selectbox("Crop", SUPPORTED_CROPS)
    planting_date  = st.date_input("Planting date", value=datetime.now())
    submitted      = st.form_submit_button("Add Plot")

if submitted:
    try:
        lat, lon = resolve_location(location_input)
        new_plot = FarmPlot(plot_name, lat, lon, crop_choice)
        new_plot.set_planting_date(datetime.combine(planting_date,
datetime.min.time()))
        st.session_state.plots.append(new_plot)
        st.success(f"Added '{plot_name}' at {lat:.4f}, {lon:.4f}.")
    except ValueError as e:
        st.error(f"Could not add plot: {e}")

```

## Section 2 — Displaying Plots and Getting Advice:

Each farm plot is shown in a collapsible `st.expander` widget. When the user clicks 'Get weather & advice', three things happen in sequence: `WeatherClient` fetches the forecast, `Crop.load()` reads the crop data, and `PlantingAdvisor` generates advice. The results are stored in `session_state` under unique keys so they persist when the user interacts with a different plot.

*app.py — unique per-plot session keys*

```

for i, plot in enumerate(st.session_state.plots):
    with st.expander(f"{plot.plot_name} — {plot.crop_name.title()}"):

        weather_key = f"weather_{i}"    # unique key for this plot's weather
        advice_key  = f"advice_{i}"     # unique key for this plot's advice

        if st.button("Get weather & advice", key=f"fetch_btn_{i}"):
            crop      = Crop.load(plot.crop_name)
            weather   = st.session_state.weather_client.get_forecast(plot.latitude,
plot.longitude)

```

```
        advice = st.session_state.advisor.get_advice(crop, weather,
datetime.now().month)

        st.session_state[weather_key] = weather    # store so it survives reruns
        st.session_state[advice_key] = advice
```

Without unique keys like `weather_0`, `weather_1`, etc., opening plot 2 would overwrite plot 1's results in memory. The f-string `{c(f"weather_{i}")}` generates a unique string per plot.



# Chapter 5: The Four Core Python Concepts

This project was designed to demonstrate four specific Python programming concepts. Here they are explained in full, with where to find them in the code.

## 5.1 Object-Oriented Programming (OOP)

OOP is the practice of organising code around objects — things that combine data (attributes) and behaviour (methods) in one place. This project has six classes:

| Class           | File                         | What It Represents                   | Key Methods                                     |
|-----------------|------------------------------|--------------------------------------|-------------------------------------------------|
| Crop            | models/crop.py               | A crop and its agricultural facts.   | load(), is_in_planting_season()                 |
| FarmPlot        | models/farm_plot.py          | One plot of farmland.                | log_activity(), to_dict(), from_dict()          |
| SeasonCalendar  | models/season_calendar.py    | Timeline and risk engine for a plot. | generate_milestones(), check_weather_warnings() |
| WeatherClient   | services/weather_client.py   | API client for weather forecasts.    | get_forecast(), _parse_response()               |
| PlantingAdvisor | services/planting_advisor.py | Decision engine for planting advice. | get_advice(), _rule_based_advice()              |
| FarmLogStore    | services/farm_log_store.py   | Disk read/write for FarmPlot data.   | save(), load()                                  |

**@classmethod:** Used in Crop.load() and FarmPlot.from\_dict(). A classmethod is an alternative constructor — instead of calling MyClass() and then setting attributes manually, you call MyClass.from\_data() and it builds the object for you. The parameter is cls (the class itself) instead of self (an instance).

**\_\_repr\_\_:** Implemented in Crop and FarmPlot. Controls what Python displays when you print or inspect an object. 'Crop(Maize)' is far more useful than ''.

## 5.2 Exception Handling

---

Exception handling lets the program respond gracefully to problems instead of crashing with a traceback that the user cannot understand. The pattern used throughout this project is `try / except / raise`:

*Exception Handling Pattern (used everywhere)*

```
try:
    response = requests.get(url, timeout=10)    # attempt the risky operation
    response.raise_for_status()                # raise error if 4xx or 5xx
response
except requests.exceptions.Timeout:
    raise TimeoutError("Server took too long.") # translate to a clean type
except requests.exceptions.ConnectionError:
    raise ConnectionError("No internet connection.")
except requests.exceptions.HTTPError:
    raise ValueError("Server rejected the request.")
```

This translation layer is important. The rest of the app only needs to know about three types: `TimeoutError`, `ConnectionError`, `ValueError`. It does not need to import the `requests` library to handle its exceptions.

## 5.3 File Handling (I/O)

---

File handling is demonstrated in three places:

- **Reading:** `crop_knowledge.json` is read at startup by `Crop.load()` using `json.load()`.
- **Writing:** `FarmLogStore.save()` writes plot data to `farm_plots.json` using `json.dump()`.
- **Reading back:** `FarmLogStore.load()` reads `farm_plots.json` and reconstructs `FarmPlot` objects.

The **`with open(...) as f`** pattern is always used. This is a context manager — it guarantees the file is closed automatically when the block ends, even if an exception is raised inside it.

## 5.4 Regular Expressions

---

Regular expressions (regex) are used in `validators.py` to check user input before it is processed. All patterns are pre-compiled with `re.compile()` for efficiency.

| Pattern Element                    | Meaning                                                                                        |
|------------------------------------|------------------------------------------------------------------------------------------------|
| <code>^</code> and <code>\$</code> | Start and end of string — the match must cover the entire input, not just part of it.          |
| <code>\d</code>                    | Any digit (0–9).                                                                               |
| <code>\d+</code>                   | One or more digits.                                                                            |
| <code>(?:\.(d+))?</code>           | Optional non-capturing group: a literal dot followed by one or more digits (the decimal part). |
| <code>-?</code>                    | Optional minus sign (for negative coordinates or temperatures).                                |
| <code>\s*</code>                   | Zero or more whitespace characters.                                                            |
| <code>[A-Za-z\s]</code>            | Character class: any uppercase letter, lowercase letter, or whitespace.                        |
| <code>(0[1-9] 1[0-2])</code>       | Alternation: month 01–09 OR 10–12.                                                             |

## Chapter 6: How All the Pieces Connect

---

The project is built in layers. Each layer depends on the ones below it, and the ones above it do not know the details of how lower layers work — they only use the public interface (the method names).

| Layer           | Files                                                     | Depends On                            |
|-----------------|-----------------------------------------------------------|---------------------------------------|
| UI Layer        | app.py                                                    | All services, all models, all utils.  |
| Service Layer   | weather_client.py, planting_advisor.py, farm_log_store.py | Models, utils, config, external APIs. |
| Model Layer     | crop.py, farm_plot.py, season_calendar.py                 | data/crop_knowledge.json, utils.      |
| Utilities Layer | validators.py, logger_setup.py                            | Python standard library only.         |
| Config Layer    | config.py, .env                                           | python-dotenv, os.                    |

**Why this layering matters:** If Open-Meteo changes their API format tomorrow, only `weather_client.py` needs to change. The rest of the app does not care how the weather dict was obtained — only what shape it is.

# Chapter 7: The Data Flow Step-by-Step

---

## Scenario: A farmer adds a plot 'North Field' in Abuja growing Maize.

Step 1 — Form submission: User clicks 'Add Plot'.

`app.py` calls `resolve_location('Abuja')`. Since 'Abuja' has no digits, it goes to the geocoding path.

Step 2 — Geocoding:

`_geocode_open_meteo('Abuja')` fires a GET to `geocoding-api.open-meteo.com`.

Gets back `{latitude: 9.0579, longitude: 7.4951}`.

Step 3 — FarmPlot creation:

`FarmPlot('North Field', 9.0579, 7.4951, 'maize')` is created.

`set_planting_date()` stores the chosen date.

The plot is appended to `st.session_state.plots`.

Step 4 — 'Get weather & advice' clicked:

`Crop.load('maize')` opens `crop_knowledge.json` and builds a Crop object.

`WeatherClient.get_forecast(9.0579, 7.4951)` checks cache — miss on first call.

GET

`https://api.open-meteo.com/v1/forecast?latitude=9.0579&longitude=7.4951&daily=...`

`_parse_response()` converts the raw JSON into the clean forecast dict.

Result stored in cache with timestamp.

Step 5 — Advice generation:

`PlantingAdvisor.get_advice(crop, weather, 7)` is called.

`_build_prompt()` assembles the text prompt with crop facts + weather numbers.

`genai.Client.models.generate_content(model='gemini-3.5-flash', ...)` is called.

If it raises any exception, `_rule_based_advice()` runs instead.

Step 6 — Calendar generation:

`SeasonCalendar(plot, crop).generate_milestones()` adds `timedeltas` to the planting date.

`check_weather_warnings()` loops through the 7-day forecast checking thresholds.

#### Step 7 — Save:

User clicks 'Save all plots to file'.

`FarmLogStore.save(plots)` calls `plot.to_dict()` for each plot.

`json.dump()` writes the list to `data/saved_plots/farm_plots.json`.

#### Step 8 — Next session:

App starts. `FarmLogStore.load()` reads `farm_plots.json`.

`FarmPlot.from_dict()` rebuilds each plot including its activity log.

Plots appear immediately in the UI without the user doing anything.

# Chapter 8: Likely Questions & Model Answers

---

## **Q: What is the purpose of this project?**

This application helps farmers make data-driven planting decisions. A farmer enters their location and chosen crop, and the app fetches live weather forecasts, compares them to the crop's requirements, and uses AI to generate personalised advice. It also generates a season milestone calendar and warns about dangerous weather conditions.

## **Q: What is OOP and where do you use it?**

Object-Oriented Programming organises code into classes that combine data and behaviour. I have six classes: Crop holds crop facts and can check if the current month is a planting season; FarmPlot stores a plot's location, crop, and activity log; SeasonCalendar generates milestone dates and weather warnings; WeatherClient fetches and caches forecast data; PlantingAdvisor generates advice using AI or a rule-based fallback; FarmLogStore reads and writes plot data to JSON files.

## **Q: What is a classmethod and why did you use it?**

@classmethod creates an alternative constructor. Instead of creating an empty object and setting attributes one by one, you call Crop.load('maize') and the method handles reading the JSON file and building the object. The parameter is cls (the class) instead of self (an instance). I use it in Crop.load() and FarmPlot.from\_dict().

## **Q: What is exception handling and how did you use it?**

Exception handling is a way to respond gracefully to errors instead of crashing. I wrap risky operations (API calls, file reads) in try/except blocks. For network calls I catch requests.exceptions.Timeout (server too slow), requests.exceptions.ConnectionError (no internet), and requests.exceptions.HTTPError (server rejected the request). I re-raise these as cleaner built-in types like TimeoutError and ValueError so the UI only needs to catch three simple types.

### **Q: What is file handling and where is it in the code?**

File handling is reading from and writing to disk. In `Crop.load()`, I use `open(filepath, 'r')` with `json.load()` to read the crop knowledge base. In `FarmLogStore.save()`, I use `open(filepath, 'w')` with `json.dump()` to write plot data. In `FarmLogStore.load()`, I read the saved file back. I always use `'with open(...) as f'` which is a context manager that closes the file automatically even if an error occurs.

### **Q: What are regular expressions and where do you use them?**

Regular expressions are patterns that describe what a valid string should look like. I use them in `validators.py` to check user input before processing it. For coordinates, the pattern checks for optional minus sign, digits, optional decimal, comma, then the same again. For location names, it only allows letters, spaces, and punctuation (no digits). For dates, it checks YYYY-MM-DD format with valid month (01-12) and day (01-31) ranges. All patterns are pre-compiled with `re.compile()` for performance.

### **Q: Why does the app have a rule-based fallback for advice?**

The rule-based fallback exists so the app always gives the farmer an answer, even if Gemini is unavailable. Reasons Gemini might fail include: no API key, no internet connection, rate limits, or a Google service outage. The fallback uses simple direct comparisons against the crop's data — if max temperature exceeds the ideal max and the crop is heat-sensitive, it warns about that. It is less nuanced than AI but always works offline.

### **Q: What is caching and why did you implement it?**

Caching stores the result of an expensive operation so it does not need to be repeated. In `WeatherClient`, every successful API response is stored in a dictionary (`self._cache`) alongside a timestamp. The next time the same coordinates are requested, the code checks the age — if less than 60 minutes old, it returns the stored data instead of making a new HTTP request. This makes the app faster and avoids overloading the free Open-Meteo API.



### **Q: What is Streamlit session\_state?**

Streamlit reruns the entire `app.py` script from top to bottom every time a user interacts with the page. Without `session_state`, any object created during a run would be lost on the next rerun. `st.session_state` is a persistent dictionary that survives reruns. I store `WeatherClient`, `PlantingAdvisor`, `FarmLogStore`, and the plots list in `session_state` so they are created once per browser session and reused on every subsequent rerun.

### **Q: Why does the geocoding use two APIs?**

No single free geocoding service has complete coverage everywhere. The primary geocoder is Open-Meteo's geocoding API, which is fast and closely aligned with the weather API. If it returns no results (for example, for a small neighbourhood or village), the app automatically tries OpenStreetMap's Nominatim service, which uses community-contributed map data and often has better coverage of smaller places. If both fail, the user can also type raw GPS coordinates directly.

### **Q: What does `to_dict()` and `from_dict()` do?**

JSON files cannot store Python objects or datetime values directly. `to_dict()` converts a `FarmPlot` into a plain Python dictionary, converting all datetime objects to 'YYYY-MM-DD' strings using `strftime()`. `from_dict()` does the reverse when loading from disk — it reads the string dates and converts them back to datetime objects using `strptime()`. This pair of methods is the bridge between Python objects in memory and JSON data on disk.

### **Q: What is the purpose of the `.env` file?**

The `.env` file stores secret values — in this case the Gemini API key — outside of the source code. If the key were hardcoded in `config.py`, it would be visible to anyone who sees the code (for example on GitHub). The `.gitignore` file ensures `.env` is never committed to version control. `python-dotenv`'s `load_dotenv()` reads the `.env` file and loads its values into the environment, where `os.getenv()` can retrieve them safely.

**Q: What is the difference between models/ and services/?**

Models (`crop.py`, `farm_plot.py`, `season_calendar.py`) are blueprints for things — data containers with methods that operate on their own data. They do not call external APIs.

Services (`weather_client.py`, `planting_advisor.py`, `farm_log_store.py`) handle interactions with the outside world: APIs, disk, the AI model. This separation means you could swap out the weather provider by only changing `weather_client.py`, without touching any model.

# End of Report

---

This document covers every file, every class, every method, and every design decision in the Smart Farming & Crop Planting Advisor project. You should be able to open any file, point to any line, and explain exactly what it does and why it is there.

*Good luck with your presentation.*